SECURITY MODULE

Apache Kafka

ACLs & Authorization

Access Control · Authorization Model · Best Practices

Rajesh Pasham

Apache Kafka Training Program | Security Module | 2026

Module Agenda

1 Security Architecture

3 Pillars · Auth Flow · Default Posture

2 ACL Data Model

Principal · Resource · Operation · Permission

3 Configuring Authorizer

AclAuthorizer · KRaft · super.users

4 Managing ACLs via CLI

kafka-acls.sh · Create · List · Remove

5 Component ACL Sets

Producer · Consumer · Connect · Streams

6 Resource Patterns

LITERAL · PREFIXED · Wildcard · Precedence

7 Troubleshooting

Error Signatures · Log Analysis · Workflow

8 Best Practices

Least Privilege · Naming · Lifecycle · Audit

Learning Objectives

- Understand Kafka's 3-pillar security architecture (Encryption · Authentication · Authorization)
- Master the ACL data model: Principal, Resource, Operation, Permission
- Configure AclAuthorizer / StandardAuthorizer and broker security settings
- Create, list, describe, and delete ACLs using kafka-acls.sh CLI
- Implement correct ACL sets for Producers, Consumers, Connect, and Streams
- Apply LITERAL, PREFIXED, and Wildcard resource patterns strategically
- Diagnose authorization failures from error messages and broker logs
- Apply ACL best practices for multi-tenant production environments

MODULE 1

Security Architecture

Encryption · Authentication · Authorization

The Three Security Pillars

1 Encryption

Can data be read in transit?

TLS / SSL

Protects data as it flows over the wire between clients and brokers, and between brokers. Without this, all messages are visible to network sniffers.

2 Authentication

Who is this client?

SASL · mTLS

Verifies client identity using SASL/PLAIN, SCRAM-SHA-256/512, GSSAPI (Kerberos), or mutual TLS certificates. Without this, every client is ANONYMOUS.

3 Authorization

Is this client allowed?

ACLs (this module)

Decides what each authenticated identity is permitted to do. ACLs only work meaningfully when authentication is enabled — otherwise all clients share ANONYMOUS identity.

How Kafka Processes a Client Request

Authorization is the LAST step — authentication must succeed first

1 **Client connects via network listener**

`PLAINTEXT:9092 or SASL_SSL:9093`

2 **Broker authenticates the client**

`SASL (PLAIN/SCRAM/Kerberos) or mTLS certificate`

3 **Broker identifies the Principal**

`e.g. User:alice or User:CN=data-pipeline,OU=Eng`

4 **ACL check: Principal + Operation + Resource**

`Authorizer evaluates all matching ACL rules`

5 **Allow or Deny**

`ALLOW → proceed | DENY / no ACL → AUTHORIZATION_FAILED`

MODULE 2

The ACL Data Model

Principal · Resource · Operation · Permission

Anatomy of a Kafka ACL

ACL Rule = Principal + Resource + Operation + Permission (ALLOW | DENY)

Principal

```
User:alice
User:data-pipeline
User:ANONYMOUS
```

The identity of the connecting client, as authenticated by Kafka. Format: Principal-Type:Name. Case-sensitive.

Resource

```
Topic:orders
Group:order-group
Cluster
```

The Kafka object the ACL applies to: Topic, Group, Cluster, TransactionalId, or DelegationToken.

Operation

```
Read, Write
Describe, Create
Delete, All
```

The action being requested. Each resource type supports different operations — producers need Write+Describe.

Permission

```
ALLOW → grants
access
DENY → blocks access
(DENY overrides
ALLOW)
```

ALLOW grants the operation. DENY blocks it — and DENY always overrides any ALLOW for the same resource.

Principal Types & Formats

The principal name must EXACTLY match the authenticated identity — case-sensitive

SASL/PLAIN	`User:<username>`	`User:alice`	Defined in JAAS config user_ entries
SASL/SCRAM	`User:<username>`	`User:data-pipeline`	Stored in ZooKeeper or KRaft metadata
Kerberos (GSSAPI)	`User:<kerberos-principal>`	`User:kafka-producer@REALM.COM`	Full Kerberos principal including realm
mTLS (client cert)	`User:<CN from cert>`	`User:CN=data-pipeline,OU=Engineering`	CN field extracted from X.509 certificate
No authentication	`User:ANONYMOUS`	`User:ANONYMOUS`	⚠️ Production clusters must never allow this

Operations by Resource Type

TOPIC Operations	
Read	Consumers
Write	Producers
Describe	All clients (required)
Create	Admins, auto-create
Delete	Admins only
Alter	Admins only
DescribeConfigs	Monitoring tools

GROUP Operations	
Read	Join group, commit offsets
Describe	View group lag + metadata
Delete	Admins only

CLUSTER Operations	
Create	Create topics cluster-wide
Alter	Modify broker configs
IdempotentWrite	Idempotent producers
ClusterAction	Inter-broker (internal)

ALLOW vs DENY — Understanding Precedence

ALLOW

- Explicitly grants the specified operation to the principal
- Multiple ALLOW ACLs can exist for the same resource
- Standard way to grant access to application service accounts
- If NO ACL exists and allow.everyone.if.no.acl.defined=false, access is implicitly DENIED

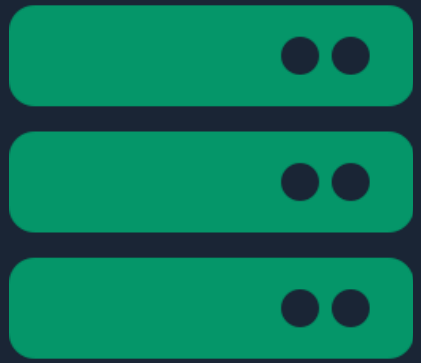
DENY

- Explicitly blocks the operation — DENY always overrides ALLOW
- If both ALLOW and DENY exist for same principal+op+resource, DENY wins
- Use sparingly — can cause confusing, hard-to-debug authorization failures
- Useful for blacklisting a specific user from a sensitive topic in a shared cluster

DENY Precedence: DENY always wins over ALLOW — regardless of ACL specificity or pattern type.

MODULE 3

Configuring the Authorizer



AclAuthorizer · StandardAuthorizer · super.users

Enabling Authorization on Brokers

ZooKeeper Mode (Kafka ≤ 3.4)

```
authorizer.class.name=
  kafka.security.authorizer.AclAuthorizer
```

KRaft Mode (Kafka 3.3+)

```
authorizer.class.name=
  org.apache.kafka.metadata
    .authorizer.StandardAuthorizer
```

Required Broker Properties

allow.everyone.if.no.acl.defined	`false`	Deny all access unless explicitly granted — MUST be false in production
super.users	`User:kafka-admin;User:broker`	These users bypass ALL ACL checks. Use semicolons, not commas
listeners	`SASL_SSL://0.0.0.0:9093`	Use authenticated listener for external clients

MODULE 4

Managing ACLs — CLI

kafka-acls.sh · Create · List · Remove

CLI Syntax & Producer ACLs

GENERAL SYNTAX

```
kafka-acls.sh --bootstrap-server <broker:port>  [ --add | --remove | --list ]
  --allow-principal User:<name>  --operation <op>  [ --topic <name> | --group <name> | --cluster ]
```

Producer ACLs — Grant Write + Describe on topic 'orders' to User:data-pipeline

```
kafka-acls.sh --bootstrap-server localhost:9092
  --add
  --allow-principal User:data-pipeline
  --operation Write --operation Describe
  --topic orders
```

Always grant both Write AND Describe to producers — without Describe, the producer cannot fetch topic metadata.

Consumer ACLs & Listing / Removing

Consumer ACLs (topic Read/Describe + group Read)

```
# Step 1: Topic permissions
kafka-acls.sh --bootstrap-server localhost:9092 \
  --add --allow-principal User:order-service \
  --operation Read --operation Describe --topic orders

# Step 2: Group permission
kafka-acls.sh --add --allow-principal User:order-service \
  --operation Read --group order-processors
```

Listing & Removing ACLs

```
# List all ACLs
kafka-acls.sh --list

# List for specific topic
kafka-acls.sh --list --topic orders

# Remove specific ACL
kafka-acls.sh --remove \
  --allow-principal User:alice \
  --operation Write --topic orders
```

No --remove-all shortcut: you must specify exact principal + operation + resource

Grant both Write AND Describe to producers; Read + Describe + Group Read to consumers

Use 'All' operation ONLY for dedicated admin accounts — never application accounts

MODULE 5

Component ACL Sets

Producer · Consumer · Kafka Connect · Kafka Streams

Complete ACL Sets — Producer & Consumer

PRODUCER Minimum ACL Set

Topic:<name>	**Write**	✓ Required
Topic:<name>	**Describe**	✓ Required
Cluster	**IdempotentWrite**	If idempotent
Cluster	**Describe**	Bootstrap

CONSUMER Minimum ACL Set

Topic:<name>	**Read**	✓ Required
Topic:<name>	**Describe**	✓ Required
Group:<gid>	**Read**	✓ Required
Group:<gid>	**Describe**	Monitoring
Cluster	**Describe**	Bootstrap

Memory Aid

Producer = Write + Describe on Topic (+ IdempotentWrite on Cluster if idempotent)
Consumer = Read + Describe on Topic + Read on Group
Both need Cluster:Describe for initial metadata bootstrap

Kafka Connect & Kafka Streams ACLs

Kafka Connect Worker ACLs

Topic:connect-configs	Read, Write, Describe, Create
Topic:connect-offsets	Read, Write, Describe, Create
Topic:connect-status	Read, Write, Describe, Create
Topic:<source-topics>	Read, Describe
Topic:<sink-topics>	Write, Describe, Create
Group:connect-*	Read, Describe
Cluster	Create, Describe

Kafka Streams ACLs

Topic:<input>	Read, Describe
Topic:<output>	Write, Describe, Create
Topic:<app-id>-*	Read, Write, Describe, Create, Delete
Group:<app-id>	Read, Describe
Cluster	Describe, Create

Use PREFIXED ACLs for Kafka Streams: --topic <app-id>- --resource-pattern-type PREFIXED covers ALL internal state store topics.

MODULE 6

Resource Patterns

LITERAL · PREFIXED · Wildcard · Precedence

Resource Pattern Types

1 LITERAL

Applies to exactly the named resource — no wildcards. This is the default.

```
--topic orders --resource-pattern-type LITERAL
```

Matches: orders ✗ Does NOT match: orders-v2, orders-archive

USE CASE

Strict per-topic ACLs for specific, known topics

2 PREFIXED

Applies to all resources whose name starts with the given prefix. Best practice for multi-tenant.

```
--topic payment- --resource-pattern-type PREFIXED
```

Matches: payment-orders, payment-refunds, payment-notifications

USE CASE

Kafka Streams internal topics, team topic namespaces, Connect groups

3 WILDCARD (*)

Setting resource name to '*' with LITERAL pattern grants op on ALL resources of that type.

```
--topic '*' --resource-pattern-type LITERAL
```

Matches: EVERY topic in the cluster

USE CASE

⚠️ Read-only monitoring only — NEVER grant Write/Delete with wildcard

Pattern Evaluation Precedence

When multiple ACL patterns match a request, Kafka evaluates in this priority order

#1

LITERAL (exact match)

Most specific — always checked first

`Topic:orders → matches 'orders' only`

#2

PREFIXED (longest match wins)

Longer prefix takes priority over shorter

`Topic:order- matches order-service, order-events`

#3

WILDCARD (*)

Least specific — checked last

`Topic:* matches all topics`

🚫 **DENY Override: Regardless of pattern priority, a DENY ACL always overrides an ALLOW ACL — even WILDCARD DENY beats LITERAL ALLOW.**

MODULE 7

Troubleshooting

Error Signatures · Log Analysis · Diagnostic Workflow

Authorization Error Signatures

Each error message maps directly to a missing ACL — know the pattern, fix it fast

TOPIC_AUTHORIZATION_FAILED	No Read or Write ACL on topic	Topic:<name> Read or Write
GROUP_AUTHORIZATION_FAILED	No Read ACL on consumer group	Group:<group-id> Read
CLUSTER_AUTHORIZATION_FAILED	No cluster-level ACL for admin op	Cluster Describe or Alter
TRANSACTIONAL_ID_AUTHORIZATION_FAILED	Missing ACL on TransactionalId	TransactionalId:<id> Write
UNKNOWN_TOPIC_OR_PARTITION	Topic missing OR no Describe ACL	Topic:<name> Describe

```
grep 'DENIED\|is not Allowed' /opt/kafka/logs/kafka-authorizer.log | tail -30
```

6-Step Diagnostic Workflow

1 Identify the Principal

Check client's sasl.jaas.config or SSL cert CN. Look in broker logs for connecting identity.

2 Reproduce the error

Note exact error code — TOPIC_AUTH_FAILED on produce = Write missing; on consume = Read or Group:Read missing.

3 List existing ACLs

kafka-acls.sh --list --allow-principal User:<name> — see what's already granted.

4 Compare vs required set

Check Module 5 tables — Producer needs Write+Describe; Consumer needs Read+Describe+Group:Read.

5 Add missing ACLs

kafka-acls.sh --add --allow-principal User:<name> --operation <op> --topic <name>

6 Verify in authorizer log

grep 'DENIED' kafka-authorizer.log — the DENIED entry should disappear after ACL is added.

Common ACL Gotchas

Case-sensitive principal	User:Alice ≠ User:alice — must match exact auth config case	Verify authentication config username casing
Missing Describe for producer	Producer fails before sending — cannot fetch topic metadata	Always add Describe alongside Write
Missing Group ACL	Consumer joins group but cannot commit offsets → GROUP_AUTH_FAILED	Add Group:<group-id> Read ACL separately
DENY shadowing ALLOW	A broad DENY ACL silently blocks a specific ALLOW — hard to debug	Remove conflicting DENY or narrow its scope
super.users wrong format	super.users=User:admin,User:broker uses commas — IGNORED	Must use semicolons: User:admin;User:broker
ACL not taking effect (ZK)	ZooKeeper-based clusters cache ACLs — refresh takes up to 30s	Wait or restart broker; KRaft is near-instant

MODULE 8

Best Practices

Least Privilege · Naming · Lifecycle · Audit

ACL Best Practices

Principle of Least Privilege

Never use 'All' for app accounts. Separate service accounts for producers, consumers, and admins. Never share credentials between apps.

Naming Conventions

Service accounts: team-service-env (e.g. payments-producer-prod). Groups: team-service-purpose-env. Topics: team.domain.event.

Infrastructure as Code

Store all ACL definitions in Git. Use Terraform Kafka provider or Confluent Ansible to manage ACLs declaratively — no manual ZK edits.

PREFIXED for Scalability

Use PREFIXED ACLs for Kafka Streams internal topics and multi-tenant team namespaces. Reduces ACL count and avoids ACL sprawl.

Regular Auditing

Monthly: list + review stale ACLs. Weekly: check DENIED in authorizer log. Quarterly: verify super.users is minimal. Remove ACLs on offboarding.

Multi-Tenant Strategy

Assign team-based topic prefixes. PREFIXED ACL per team. allow.everyone.if.no.acl.defined=false enforces isolation automatically.

Recommended Naming Conventions

Service Accounts	`team-service-env`	`payments-producer-prod` `analytics-consumer-dev`	*Separate per role & environment*
Consumer Groups	`team-service-purpose-env`	`payments-order-listener-prod` `analytics-events-processor-dev`	*Unique per purpose*
Topic Prefixes	`team.domain.event`	`payments.orders.created` `analytics.pageviews.raw`	*Use dots as namespace separators*
Admin Accounts	`kafka-admin-env`	`kafka-admin-prod` `kafka-admin-dev`	*Never use for applications*

Consistent naming enables PREFIXED ACLs to work elegantly — one ACL entry covers an entire team's topic namespace.

HANDS-ON

6 Lab Exercises

Beginner · Intermediate · Advanced · Challenge

Lab Overview

1	**Enable ACLs & Verify the Authorizer** Confirm authorizer is active; verify unauthenticated client is denied	20 min	Beginner
2	**Producer ACL — Grant & Verify Write Access** Grant alice Write+Describe on 'orders'; verify produce; revoke and re-verify	25 min	Beginner
3	**Consumer ACL — Topic + Group Access** Grant Read+Describe on topic + Group:Read to bob; verify consumer offset commits	30 min	Intermediate
4	**PREFIXED ACLs for Multi-Tenant Topics** Grant PREFIXED ACL on team-a- prefix; verify only team-a topics are accessible	30 min	Intermediate
5	**Troubleshoot Authorization Failures** Diagnose 4 real-world auth failure scenarios using error logs and ACL lookup	35 min	Intermediate
6	**Challenge: Secure a Full Application Stack** Design + implement complete ACL policy for 6-service e-commerce platform	40 min	Advanced

Total Lab Time: ~3 hours | Prerequisites: Kafka with SASL/PLAIN + ACL authorizer configured

Security Module Complete!

ACLs & Authorization

ACL = Principal + Resource + Operation + Permission

DENY always overrides ALLOW — use sparingly

Least privilege: separate accounts per role

PREFIXED ACLs for Streams, Connect, multi-tenant

Rajesh Pasham